from the host, mount it in the UML instance, and copy the modules.

Note:

- This set-up assumes that there is a running DHCP server in the host's network. If this is not the case, interface *br0* on the host and *eth0* in the UML guest have to be assigned static IP addresses. We need to remember that the UML system lies in the same network as the host system, in this set-up.
- We follow this model of setting up the network because if administrators want to provide sandboxed UML test environments for users who need full privileges, they would either need the UML instances to be on the same network as the host, or would need to configure a custom *iptables* set-up.

Debugging the Linux kernel in UML

Since UML is considered to be an application, it can be debugged with the standard GDB debugger, as follows.

Load the *linux* binary in GDB:

```
$ gdb linux-2.6.35.rc3/linux
```

This gives us a *gdb* prompt. Since we could not specify the command-line arguments for the UML instance on the GDB command-line, we set the arguments here (the *eth0* argument is assuming that you have set up bridged network access between the host and the UML instance, as described above):

```
$ set args ubda=FedoraCore6-AMD64-root_fs
mem=250M eth0=tuntap,,tap1
```

Figure 7 illustrates the UML kernel being booted from within GDB. Once we passed the arguments with the *set args* command, we placed a breakpoint on the *start_kernel()* function in the kernel code, and then instructed GDB to run the program. As you can see, after UML initialisation, GDB stopped execution when it reached the breakpoint.

If you did not start UML in the GDB debugger, you can also attach GDB to the UML guest later:

```
# gdb linux-2.6.35-rc3/linux 2666
```

(Here, 2666 is the PID of the UML instance. See Figure 8 for an illustration of attaching GDB to a running UML instance.)

Compiling custom modules for UML

If you have written a custom kernel module that you need to insert into the running UML kernel, the module needs to be compiled for the UML architecture, with the same kernel version with which UML is running.

Your *Makefile* could look like what's shown below:

```
obj-m := uml-mod.o
      KPATH := /code/kernel/lfy/mods/lib/
modules/2.6.35-rc3/build
      PWD := $(shell pwd)
all:
      $(MAKE) -C $(KPATH) SUBDIRS=$(PWD) modules
```

Here, *KPATH* defines the path of the UML kernel source. Remember to pass *ARCH=um* with the *make* command:

```
# ARCH=um make
```

This will compile your custom kernel module for the UML kernel. Once the module is compiled successfully, you can copy the '.ko' file from the host system to the UML using hostfs or networking, as given above, and you can then insert it into the running UML kernel.

Debugging modules with UML

Loadable modules are a great advantage in the Linux kernel. Pieces of kernel code can be dynamically plugged in and out of the running kernel. However, a